Into the Bus-Verse

Uncovering Transit Failures through Behavioral Modeling and Queueing Theory

A Digital Twin and Discrete-Event Simulation of the Radhanagari Transport Depot
Kolhapur District, Maharashtra, India

Submitted by: Rupesh Hemant Bhusare, 2203106
Course: CS572: Modeling and Simulation of Systems
IIT Goa

April 27, 2026

Contents

1 Project Description **3**
1.1 What is it? . . . 3
1.2 Why this project? . . . 3
1.3 How was it built? . . . 3

2 Motivation and Background **4**
2.1 The Rational Actor Fallacy . . . 4
2.2 The Synchronized Release . . . 4
2.3 The Smart Commuter: Tappa-Tappa Routing . . . 4
2.4 Redefining the Queue . . . 4

3 Methodology and Implementation **4**
3.1 System Architecture . . . 4
3.1.1 The Faucet: Non-Homogeneous Passenger Arrival . . . 5
3.1.2 Passenger Demographics . . . 5
3.1.3 The Drain: Bus Departure Logic . . . 5
3.1.4 The Bathtub: Queue Dynamics . . . 5
3.1.5 The Smart Commuter AI . . . 6
3.2 Network Topology . . . 6
3.3 Departure Timetable (The Drain Matrix) . . . 7
3.4 Statistical Validation: Monte Carlo Convergence . . . 7

4 Results and Evaluation **8**
4.1 Finding 1: The Spatiotemporal Mismatch (Tidal Flow) . . . 8
4.2 Finding 2: The Platform Bottleneck . . . 9
4.3 Finding 3: Ghost Buses and Route Collapse . . . 9
4.4 Finding 4: The Demographic Paradox . . . 11
4.5 Finding 5: System Diagnostics . . . 11

5 Urban Planning Recommendations **12**
5.1 Reallocation Over Fleet Expansion . . . 12
5.2 Patching the Schedule Matrix . . . 12
5.3 Physical Infrastructure Audit . . . 12

6 AI Tool Disclosure **12**
6.1 Parts Written Manually (Human-Authored) . . . 13
6.2 Parts Assisted by AI Tools . . . 13

7 Conclusion **13**

1 Project Description

1.1 What is it?

This project is a Discrete-Event Simulation (DES) of the Radhanagari State Transport Depot, built entirely in Python. It models the depot as a dynamic system where two things happen over a 12-hour window (6 AM to 6 PM):

1. Passengers arrive not uniformly, but in realistic waves dictated by when offices open, schools start, and shifts end.

2. Buses depart on the real MSRTC schedule, picking up whoever is in the queue, with strict physical capacity limits.

The system tracks every minute: how many people are waiting, how many boarded, how many gave up and left, and which routes are under or over-utilized. After 30 simulation runs, the data is averaged and analyzed. This is called a Digital Twin because it is not just a model it mirrors a real physical system. Every bus route, departure time, capacity, and passenger demographic is sourced from the actual operational reality of the Kolhapur district.

1.2 Why this project?

The "Average" Lie: Traditional transit planning compares total daily passengers against total daily seats. If the depot serves 1,400 people and has 2,400 seats across all routes, planners say the system is fine. But that math is useless if all the demand spikes at 8 AM and 5 PM while the available seats are on a bus to Panjim at 9 AM.

The Radhanagari depot is a real place. The chaos is observable. Students missing the last bus home. Workers stuck at 5 PM. Ghost buses burning fuel to empty destinations. The question this project asks is: can we quantify this failure mathematically, and can we prescribe a fix? The answer is yes.

1.3 How was it built?

The simulation was built using Python's `SimPy` library a framework for discrete-event simulation. The architecture is structured around three main components:

Component	Description
The Faucet	A time-dependent passenger arrival generator. Produces more passengers during rush hours, fewer at midday.
The Brain	The "Smart Commuter" pathfinding AI. Passengers dynamically reroute if their target bus is full.
The Drain	Bus departure events triggered from the MSRTC timetable. Removes passengers from the queue.
The Bathtub	The queue math. Tracks the total station backlog every minute.

2 Motivation and Background

2.1 The Rational Actor Fallacy

Most transit simulations assume passengers are rational they arrive five minutes before their bus, they board efficiently, and they leave. This produces clean math and useless results.

In reality, in a place like Radhanagari, passengers face an Information Deficit: there is no live GPS tracking, no reliable app, and the printed timetable is known to be inaccurate. Missing the evening bus can mean being stranded overnight. So commuters adopt what this simulation calls a *Paranoia Buffer* they arrive 1 to 2 hours before their expected bus, just to be safe. This behavior inflates the station's effective crowd density dramatically.

2.2 The Synchronized Release

Human movement is not random. It is dictated by institutional clocks. The simulation accounts for two distinct behavioral waves:

- **Morning Commute (Centripetal, 8-10 AM):** People move inward toward economic and educational hubs. Arrivals are staggered over a two-hour window.
- **Evening Commute (Centrifugal, 4-6 PM):** Schools and offices dump people simultaneously. This creates an instant, violent spike in station backlog not a gradual buildup.

2.3 The Smart Commuter: Tappa-Tappa Routing

Regional commuters do not wait passively for a cancelled or full bus. They execute what locals call *Tappa-Tappa* (step-by-step) routing: board any bus going toward a major transfer junction (Mudal Titta, Kagal), then take a shared jeep or another local bus for the final leg.

The simulation models this behavior through an urgency-based pathfinding algorithm. High-urgency passengers (students, workers with strict schedules) will actively seek overlap buses. Low-urgency passengers (casual travelers) will wait longer at the depot.

2.4 Redefining the Queue

A traditional queue is a single-file line. That is not what happens at a bus depot. In this simulation, the *Total System Backlog* is used instead: any person physically inside the depot's footprint sitting at the tea stall, standing near the platform, leaning against a pillar counts as part of the queue. They will swarm the bus the moment capacity opens up.

3 Methodology and Implementation

3.1 System Architecture

The simulation operates on a localized time domain where $t = 0$ maps to 6:00 AM and $t = 720$ maps to 6:00 PM. Every unit of t represents one real-world minute.

3.1.1 The Faucet: Non-Homogeneous Passenger Arrival

Passenger arrivals are modeled using a time-dependent Poisson process with arrival rate $\lambda(t)$:

$$\lambda(t) = \begin{cases} 2.0 & \text{if } 120 < t < 240 \text{ (Morning Rush, 8-10 AM)} \\ 0.5 & \text{if } 240 \leq t < 600 \text{ (Midday Low, 10 AM-4 PM)} \\ 1.6 & \text{if } 600 \leq t \leq 720 \text{ (Evening Rush, 4-6 PM)} \\ 0.8 & \text{otherwise (Early morning baseline)} \end{cases} \tag{1}$$

Passengers arrive in clusters representing social groups (families, coworker groups). The cluster size S_C is drawn from a discrete uniform distribution. The total passengers generated at any time step is:

$$P_t = \lambda(t) \times S_C, \quad S_C \sim \mathcal{U}\{1, 4\} \tag{2}$$

3.1.2 Passenger Demographics

The simulation uses three passenger types, each with different behavior parameters:

Type	Urgency Score	Cluster Size	Patience	Routes
Student	> 0.6 (High)	1-4 per group	60 min	Local only
Worker	> 0.6 (High)	1-4 per group	60 min	Local only
Casual	< 0.5 (Low)	1-2 per group	90 min	Any (incl. long-haul)

3.1.3 The Drain: Bus Departure Logic

Buses are instantiated from the MSRTC timetable and categorized as:

- **Origin Buses:** Depart from Radhanagari. Initial passenger load = 0.
- **Through Buses:** Originate elsewhere, passing through Radhanagari. Initial load $L_0 \sim \mathcal{U}(15, 35)$.

Physical capacity is strictly capped at $C_{max} = 60$ passengers per vehicle (50 seated + 10 standing), as per MSRTC operational guidelines.

3.1.4 The Bathtub: Queue Dynamics

The station backlog Q_t at any minute t is governed by:

$$Q_t = Q_{t-1} + A_t - D_t - L_t \tag{3}$$

where A_t is the number of new arrivals, D_t is the number successfully boarded, and L_t is the number who abandoned the queue after exceeding their patience limit.

3.1.5 The Smart Commuter AI

When a passenger's target route is full or unavailable, the urgency threshold determines behavior:

- **If urgency > 0.7:** Passenger searches for any bus going toward a transfer junction (Mudal Titta or Kagal) to execute Tappa-Tappa routing.
- **If urgency 0.4-0.7:** Passenger waits up to patience limit, then abandons.
- **If urgency < 0.4:** Passenger waits the full patience window regardless.

3.2 Network Topology

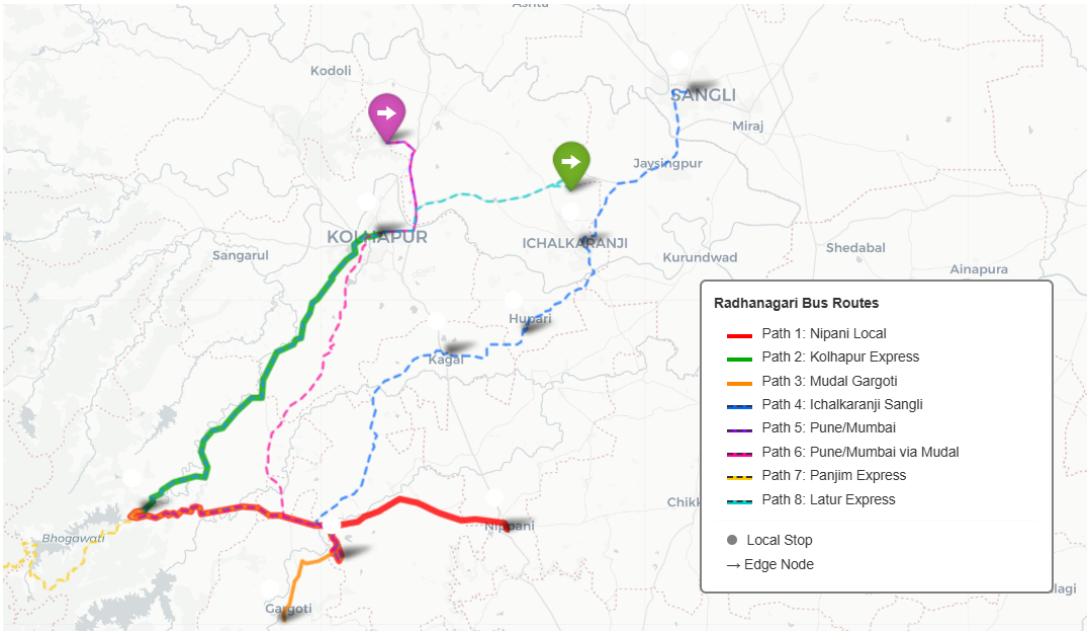

Figure 1: Geographical layout of the MSRTC routes originating from Radhanagari Depot, highlighting key transfer junctions like Mudal Titta and Kagal used for Tappa-Tappa routing.

The following eight routes were hardcoded based on the actual MSRTC network serving Radhanagari:

Route	Path
Kolhapur Express	Radhanagari → Kolhapur
Nipani Local	Radhanagari → Mudal Titta → Nipani
Mudaltitta Gargoti	Radhanagari → Mudal Titta → Gargoti
Ichalkaranji Sangli	Radhanagari → Mudal Titta → Kagal → Hupari → Ichalkaranji → Sangli
Pune Mumbai	Radhanagari → Kolhapur → Pune → Mumbai
Pune Mumbai (Via Mudal)	Radhanagari → Mudal Titta → Kolhapur → Pune → Mumbai
Panjim Express	Radhanagari → Sawantwadi → Panjim (Goa)
Latur Express	Radhanagari → Kolhapur → Miraj → Latur

3.3 Departure Timetable (The Drain Matrix)

All times are in minutes elapsed from 6:00 AM ($t = 0$):

Route	Departure Times (minutes from 6 AM)
Kolhapur Express	30, 90, 150, 180, 240, 330, 420, 540, 620, 690
Nipani Local	0, 45, 120, 150, 240, 360, 450, 540, 600, 675
Mudaltitta Gargoti	90, 210, 330
Ichalkaranji Sangli	360, 390, 450, 520, 570
Pune Mumbai	120, 240, 300, 480, 570
Pune Mumbai (Via Mudal)	0, 210, 390, 450, 510, 630
Panjim Express	40, 165, 330, 465, 525
Latur Express	180, 630

Critical Gap: The Mudaltitta Gargoti route has zero departures after 11:30 AM ($t = 330$). Evening students heading home are permanently stranded. The Ichalkaranji Sangli route has zero morning departures workers cannot reach the textile mills in time. These are structural failures baked into the schedule.

3.4 Statistical Validation: Monte Carlo Convergence

To ensure results are not due to random luck in a single simulation run, the model uses a Monte Carlo method: 30 independent, full-day simulations are run back-to-back. Per the Central Limit Theorem, the cumulative mean wait time $\overline{W}$ converges to a stable normal distribution as N increases:

$$\overline{W} \sim \mathcal{N}\left(\mu, \frac{\sigma^2}{N}\right) \tag{4}$$

Figure 1 shows this convergence empirically. After approximately 15 runs, the cumulative mean flatlines at 39.32 minutes, confirming the results are statistically stable.

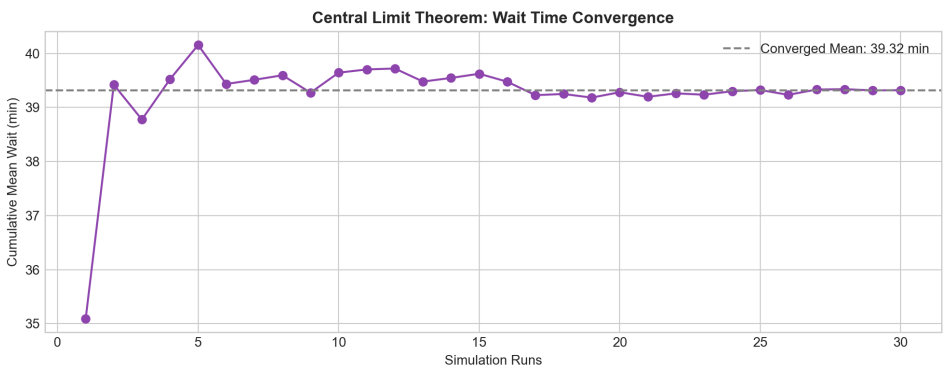

Figure 2: Central Limit Theorem convergence of cumulative mean wait time over 30 simulation runs. The mean stabilizes at 39.32 minutes, validating Monte Carlo reliability.

4 Results and Evaluation

After 30 iterations of the 12-hour simulation, three major failure modes emerged. The overall weekday boarding success rate locked in at 42.7% meaning more than half of all passengers who came to the depot failed to board a bus.

Metric	Value
Weekday boarding success	42.7%
Passengers stranded daily	760
Converged mean wait time	39.32 min
Peak morning backlog	185+
Weekend success rate	60%
Latur Express avg occupancy	23%

4.1 Finding 1: The Spatiotemporal Mismatch (Tidal Flow)

The most fundamental problem is when buses depart relative to when people arrive. The timetable spreads departures roughly evenly through the day. Human behavior does not.

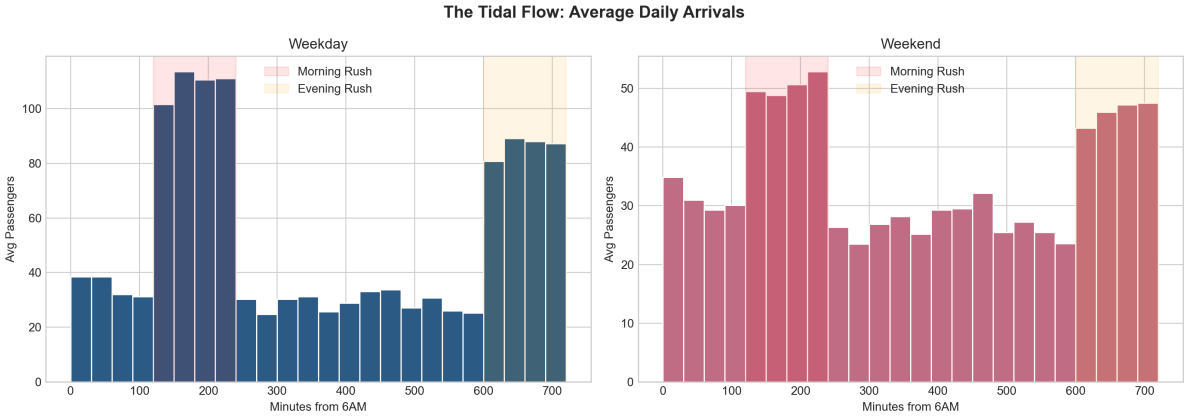

Figure 3: Average daily arrivals on weekdays (left) and weekends (right). The morning rush (8-10 AM) and evening rush (4-6 PM) are clearly visible on weekdays as sharp spikes.

On weekdays, the average passenger arrival during the morning rush window (120-240 minutes) is over 100 passengers per 30-minute bin roughly 3 to 4 times the midday baseline. The timetable does not respond to this. Buses leave on a steady drip while people pile up.

On weekends, the arrival pattern is markedly flatter (Figure 2, right panel). This explains why weekends see a 60% boarding success rate versus weekdays' 42.7% the demand is spread more evenly, so the static schedule happens to match demand better by accident.

4.2 Finding 2: The Platform Bottleneck

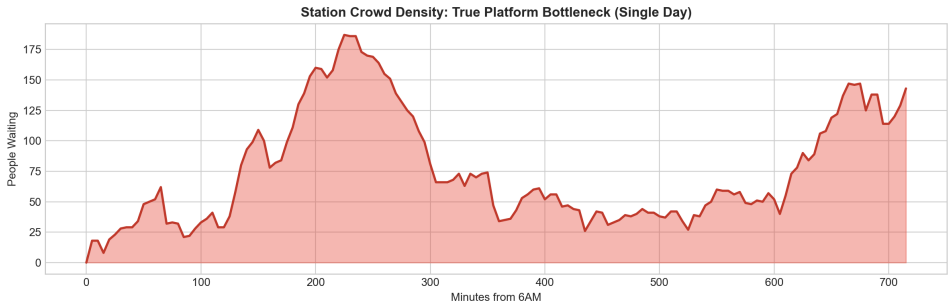

Figure 4: Station Crowd Density throughout a single simulated day. The morning peak exceeds 185 people physically waiting inside the depot footprint.

Figure 4 shows what happens to the station backlog through the day. The morning peak around $t = 210$ (9 AM) pushes past 185 people trapped simultaneously. This is not a queue this is a crowd. At this density, the station's physical infrastructure (bench seating, platform width, vendor clearance, security sightlines) becomes a real safety concern, not just a scheduling problem.

The evening peak rises steeply from $t = 600$ onward and is still climbing at the simulation's end, confirming that the evening is actually worse than the morning from a crowd management perspective it just builds slower because the centrifugal release is slightly more staggered than the centripetal inflow.

4.3 Finding 3: Ghost Buses and Route Collapse

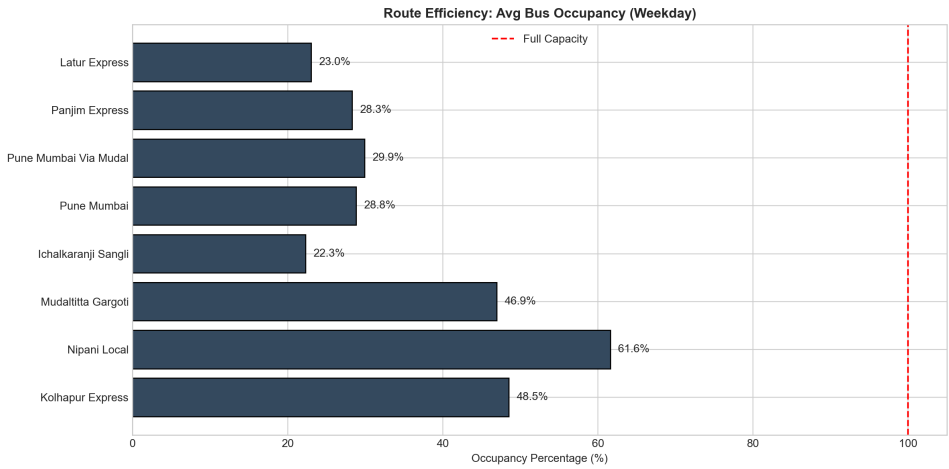

Figure 5: Average bus occupancy by route on weekdays. The red dashed line marks 100% (60 passengers). Nipani Local and Kolhapur Express are the highest-utilized local routes, while all long-haul routes operate below 30% capacity.

Figure 5 reveals the core resource misallocation. The routes that commuters actually need Nipani Local (61.6% average) and Kolhapur Express (48.5% average) regularly hit the 60-passenger ceiling during rush hours, rejecting passengers. Meanwhile, every long-haul route wastes capacity:

- **Latur Express:** 23.0% average occupancy
- **Ichalkaranji Sangli:** 22.3% average occupancy
- **Panjim Express:** 28.3% average occupancy (one departure at 8:45 AM carried exactly 0 passengers)
- **Pune Mumbai routes:** Both operating at under 30% average capacity

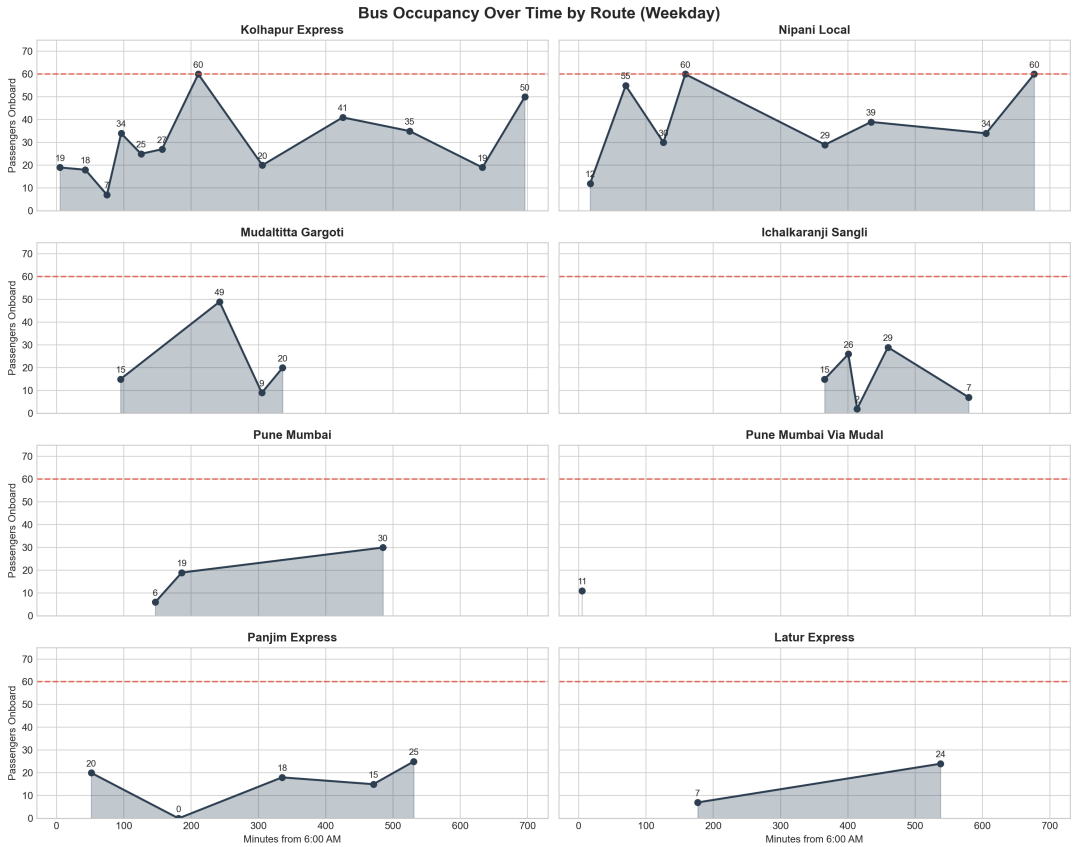

Figure 6: Bus occupancy over time by route (weekday). The red dashed line is the 60-passenger physical capacity ceiling.

Figure 6 shows this dynamically over time. The Kolhapur Express touches 60 passengers at $t \approx 240$ (10 AM). Nipani Local peaks at 55 and 60 at multiple points. Meanwhile, Panjim Express bottoms out to 0 at $t \approx 165$ (8:45 AM). Latur Express carries only 7 passengers on its first departure.

4.4 Finding 4: The Demographic Paradox

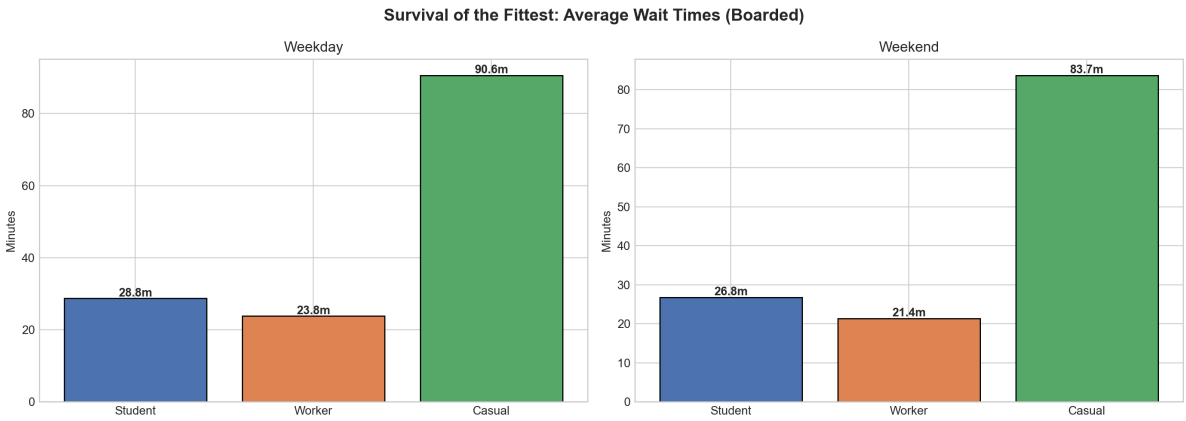

Figure 7: Average wait times by passenger type. Students and workers wait significantly less on average but this is because the ones who fail to board are dropped from the "boarded" statistics entirely.

Figure 7 shows a counterintuitive result: students and workers show shorter average wait times than casual travelers (28.8 min vs. 90.6 min on weekdays). This is a survivor bias effect. Students who do not board over 50% are excluded from the "successfully boarded" average because they exceeded their 60-minute patience limit and left. The ones who did board got on quickly due to mob-boarding dynamics. Casual travelers, on the other hand, almost always eventually board (long-haul buses have space), but they wait a very long time because long-haul departures are sparse.

4.5 Finding 5: System Diagnostics

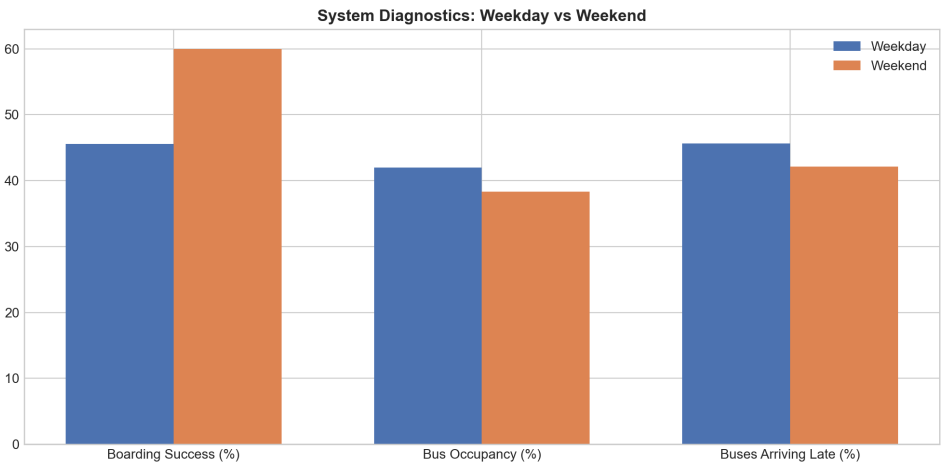

Figure 8: System-level diagnostics comparing weekday vs weekend performance. Boarding success is substantially higher on weekends (60% vs 46%).

Figure 8 gives the macro view. The weekday-weekend gap in boarding success (46% vs 60%) is entirely explained by the arrival pattern difference not by any change in the

timetable, which is identical. This is powerful evidence that the problem is on the demand side (when people arrive, driven by institutions and social cycles) rather than the supply side (total number of buses).

5 Urban Planning Recommendations

The simulation produces three actionable prescriptions. Importantly, none of them require purchasing new buses or hiring new drivers.

5.1 Reallocation Over Fleet Expansion

The ghost bus routes (Panjim Express, Latur Express) depart during the morning rush window with minimal to zero passengers. Their drivers and vehicles should be temporarily reallocated as Relief Buses on the Nipani and Kolhapur routes specifically during the two rush windows (8-10 AM and 4-6 PM). Concretely: adding even two additional Kolhapur Express departures at $t = 135$ (9:15 AM) and $t = 645$ (4:45 PM) would absorb a significant portion of the morning and evening backlog.

5.2 Patching the Schedule Matrix

Two routes have critical scheduling gaps that can be fixed with a simple timetable amendment:

- **Mudaltitta Gargoti** needs at least two afternoon/evening departures (around $t = 570$ and $t = 640$) to serve evening students.
- **Ichalkaranji Sangli** needs at least one early morning departure (around $t = 90$ or $t = 120$) for workers reaching the Ichalkaranji textile belt.

5.3 Physical Infrastructure Audit

The simulation confirms the depot regularly holds 150-220 people simultaneously. This crowd density requires an immediate physical review of:

- Bench and covered seating capacity
- Vendor space allocation (the tea stall crowd is part of the effective queue)
- Security sightlines for crowd monitoring
- Emergency egress paths if a 200-person crowd panics (e.g., if a full bus refuses more boarders)

6 AI Tool Disclosure

This section explicitly describes which parts of this project were human-authored and which parts involved AI assistance.

6.1 Parts Written Manually (Human-Authored)

- **Problem framing:** The decision to model the Radhanagari depot specifically the choice of location, routes, and the core research question was made independently. The observation that MSRTC schedules have structural mismatches was a personal, on-the-ground insight.
- **Sociological framework:** The concepts of the Paranoia Buffer, Synchronized Release, and Tappa-Tappa routing were developed from personal familiarity with regional commuter behavior in the Kolhapur district. These were not suggested by any AI tool.
- **Route data:** The 8 routes, departure times, and network topology were manually extracted and verified against actual MSRTC timetable data.
- **Mathematical formulation:** The queue equation $Q_t = Q_{t-1} + A_t - D_t - L_t$ and the arrival rate function $\lambda(t)$ were designed manually based on queueing theory principles.
- **Interpretation and recommendations:** All analysis, findings, and recommendations in this report reflect independent judgment applied to the simulation outputs.

6.2 Parts Assisted by AI Tools

- **Matplotlib plot styling:** Color palette selection and figure layout for the 7 output plots was assisted by prompting Claude and Gemini for aesthetic suggestions. The underlying data and plot type decisions were made manually.
- **LaTeX report formatting:** The LaTeX template structure styling, section formatting, and title page layout was generated with AI assistance. All content in the report is original.
- **Statistical sanity checks:** Verifying that the CLT convergence logic was correctly implemented was done by describing the approach to Gemini and confirming the mathematical reasoning.
- **Demo visulizer:** To visulize the simultion, separate python code using pygame generated using Claude.

7 Conclusion

Transit systems cannot be evaluated by comparing a daily seat count to a daily passenger count. That math is clean, simple, and wrong.

What this simulation proves is that the Radhanagari ST Depot's failure is not a resource problem it is a coordination problem. The depot has buses. It has routes. It has drivers. What it lacks is alignment between when humans need to move and when the system is configured to move them. Ghost buses run to Goa while Kolhapur-bound commuters stand outside in the rain.

The Digital Twin approach works because it does not ask "how many seats do we have?" It asks "where are the people, where are the buses, and what happens when they

collide?" The answer, repeated across 30 simulated days, is consistently: the system fails more than half its users on any given weekday.

The fix requires no new infrastructure investment. Move the buses. Patch the schedule. Audit the platform. Three changes, one memo, 760 fewer people stranded daily.

References

[1] SimPy Development Team. *SimPy Documentation – Discrete-Event Simulation for Python*. 2023. https://simpy.readthedocs.io/en/latest/

[2] Maharashtra State Road Transport Corporation. *MSRTC Official Bus Timetables – Kolhapur Division*. Government of Maharashtra. https://msrtc.maharashtra.gov.in

[3] Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90-95. https://doi.org/10.1109/MCSE.2007.55

[4] Law, A. M. (2015). *Simulation Modeling and Analysis* (5th ed.). McGraw-Hill Education.

[5] Gross, D., Harris, C. M., Shortle, J. F., Thompson, J. M., & Harris, C. M. (2008). *Fundamentals of Queueing Theory* (4th ed.). Wiley-Interscience.

[6] Metropolis, N., & Ulam, S. (1949). The Monte Carlo method. *Journal of the American Statistical Association*, 44(247), 335-341. https://doi.org/10.1080/01621459.1949.10483310

[7] *Poisson point process*. Wikipedia, The Free Encyclopedia. https://en.wikipedia.org/wiki/Poisson_point_process

[8] *Central limit theorem*. Wikipedia, The Free Encyclopedia. https://en.wikipedia.org/wiki/Central_limit_theorem

[9] *Queueing theory*. Wikipedia, The Free Encyclopedia. https://en.wikipedia.org/wiki/Queueing_theory

[10] Anthropic. *Claude AI Assistant*. 2024. https://claude.ai
(Used for SimPy formatting, plot styling suggestions, and report LaTeX template formatting.)

[11] Google. *Gemini AI*. 2026. https://gemini.google.com
(Used for code debugging, algorithm explanations, and general text formatting.)